SPECIFICATION

Calypso Terminal API

Calypso Crypto Legacy SAM

Version 2.0.0-SNAPSHOT, 2026-07-20

Warning. The present document cancels and replaces the previous release.

Link and Contact

Website
https://calypsonet.org

Support
support@calypsonet.org

Table of Contents

1. Abstract

This document is the official technical specification of the **Terminal Calypso Crypto Legacy SAM API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way, the interfaces, classes, enumerations, exceptions, structural relationships and behavioural requirements that any conforming implementation of the Terminal Calypso Crypto Legacy SAM API MUST satisfy.

The Terminal Calypso Crypto Legacy SAM API exposes the abstractions required to operate Calypso **Legacy SAM** products (SAM C1, HSM C1, S1E1, S1Dx). It is the symmetric counterpart of the **Terminal Calypso Crypto Symmetric API** (CNA-TCCS-API) for the specific case where the crypto module is a legacy Calypso SAM.

Document Status

Reference	YYMMDD-SP-CNATerminalAPI-CalypsoCryptoLegacySAM
Short name	CNA-TCCL-API
Version	2.0.0-SNAPSHOT
Revision date	2026-07-20
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-calypso-crypto-legacysam-uml-api
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)

The present document specifies the 2.0.0-SNAPSHOT of the Terminal Calypso Crypto Legacy SAM API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List

This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v2.0.0-SNAPSHOT 2026-07-20	Baseline.

2. Introduction

2.1. Purpose

The Terminal Calypso Crypto Legacy SAM API defines the public surface used by terminal applications and card-transaction managers to:

- select, unlock and identify a Legacy SAM;
- read SAM parameters, system keys, work keys and event counters;
- write counter ceilings and configurations (with or without a control SAM);
- compute and verify **basic** and **traceable** signatures using SAM keys;
- perform PKI-related operations on the SAM: key-pair generation, card-certificate computation, lock-file transfer/write;
- operate **asynchronous** write transactions through a create/execute split;
- enrich a Calypso card transaction with SAM-backed crypto operations through a card-transaction extension.

The API spans three categories of transaction managers:

Manager family	Purpose
`FreeTransactionManager`	Operations on the SAM without involving a **control SAM**—the typical "read" workflow plus signature computation and verification.
`SecureWriteTransactionManager`	Synchronous **write** operations on the SAM, requiring a **control SAM** to authorise the key/lock transfers.
`AsyncTransactionCreatorManager` / `AsyncTransactionExecutorManager`	Two-step **asynchronous** write workflow: the creator prepares the command stream using a control SAM, the executor replays it on the target SAM.

2.2. Scope

This specification covers:

- the SAM data model (`LegacySam`, `SamParameters`, `KeyParameter`);
- the selection extension used to enrich a Calypso reader selection with SAM-specific commands;
- the transaction managers, signature data carriers, key-pair and certificate computation data carriers;
- the security setting carrier and the card-transaction crypto extension;
- the SPIs implemented by the application to provide unlock data and revocation information;
- the exceptions raised by the transaction managers.

The following topics are **out of scope**:

- the on-the-wire APDU dialog with the SAM (handled internally by implementations),
- the algorithms used by the SAM to derive keys or sign data,
- the storage of certificates and key parameters outside the SAM.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises exceptions exclusively of the types defined in Section 5.5 for the situations described,
3. it preserves the structural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.3).

2.4.2. Language-agnostic types

Symbolic type	Description	Typical bindings
`string`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`boolean`	Two-state truth value.	`boolean`, `bool`.
`byte`	Signed 8-bit value.	`byte`, `i8`.
`integer`	Signed 32-bit integer.	`int`, `Int32`.
`byte array`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`date`	Calendar date without a time-of-day or timezone.	`LocalDate`, `Date`.
`sortedMap<K,V>`	Associative array sorted by key.	`SortedMap<K,V>`, `BTreeMap<K,V>`.
`T?`	Nullable value: either a `T` value or the absence of value (`null`).	`Integer`/`Byte` (boxed), `Optional<T>`, `T?`.
`void`	Indicates that an operation returns no value.	`void`, `Unit`, `None`.

2.4.3. Type tables

Each interface, class, enumeration and exception defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, exception, etc.).
Namespace	The namespace in which the type is defined.
Generics	The generic type parameter of this type, when applicable.
Extends	The type extended by this type, when applicable.

Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's language-agnostic signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Throws	A list of exceptional conditions, each mapped to a specific exception type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Non-`null` arguments.** Unless explicitly stated otherwise, every input parameter MUST be non-`null`. An implementation MUST raise `IllegalArgumentException` if a `null` value is supplied. This implicit `null` check is not repeated in the **Throws** row, which states `None.` when no other exception can occur.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.
- **Collections.** Unless the return type is marked nullable (`T?`), an operation whose return type is a collection (e.g. `list`, `set`, `map`) returns an empty collection rather than `null`.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TR-API]	*CNA Terminal API — Reader* (SP-CNATerminalAPI-Reader), version 3.0, Calypso Networks Association.
[CNA-TCC-API]	*CNA Terminal API — Calypso Card* (SP-CNATerminalAPI-CalypsoCard), version 3.0, Calypso Networks Association.
[CNA-TCCS-API]	*CNA Terminal API — Calypso Crypto Symmetric* (SP-CNATerminalAPI-CalypsoCryptoSymmetric), version 0.1, Calypso Networks Association.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816]	*Identification cards — Integrated circuit cards*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
SAM	Secure Application Module—a tamper-resistant cryptographic module used to perform Calypso operations.
Legacy SAM	A SAM compliant with the original Calypso SAM specifications (SAM C1, HSM C1, S1E1, S1Dx).
Control SAM	SAM used to authorise sensitive operations on a **target SAM** (e.g. key transfer).
Target SAM	SAM on which the operations are applied.
System Key	Key dedicated to the management of the SAM itself (personalisation, key management, reloading, authentication).
Work Key	Key used for application-level cryptographic operations.
Event Counter	Numeric counter managed by the SAM, typically used to track the usage of a work key.
Counter Ceiling	Maximum value an event counter is allowed to reach before being considered exhausted.
Unlock Data	Secret value required to unlock a locked SAM. Can be static or dynamic.
Lock File	SAM file controlling whether the SAM is locked or unlocked.
SAM Traceability	Mode in which the signature embeds the signing SAM serial number and the value of the counter associated with the signing key.
Busy Mode	Defensive mode that rate-limits repeated "PSO Verify Signature" attempts after a failure.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
CA	Certification Authority
CNA	Calypso Networks Association
HSM	Hardware Security Module
KIF	Key Identifier
KVC	Key Version Code
PKI	Public Key Infrastructure
PSO	Perform Security Operation (ISO/IEC 7816-8 command class)
SAM	Secure Application Module
SPI	Service Provider Interface
SV	Stored Value
UML	Unified Modelling Language

4. Architectural Overview

4.1. Functional positioning

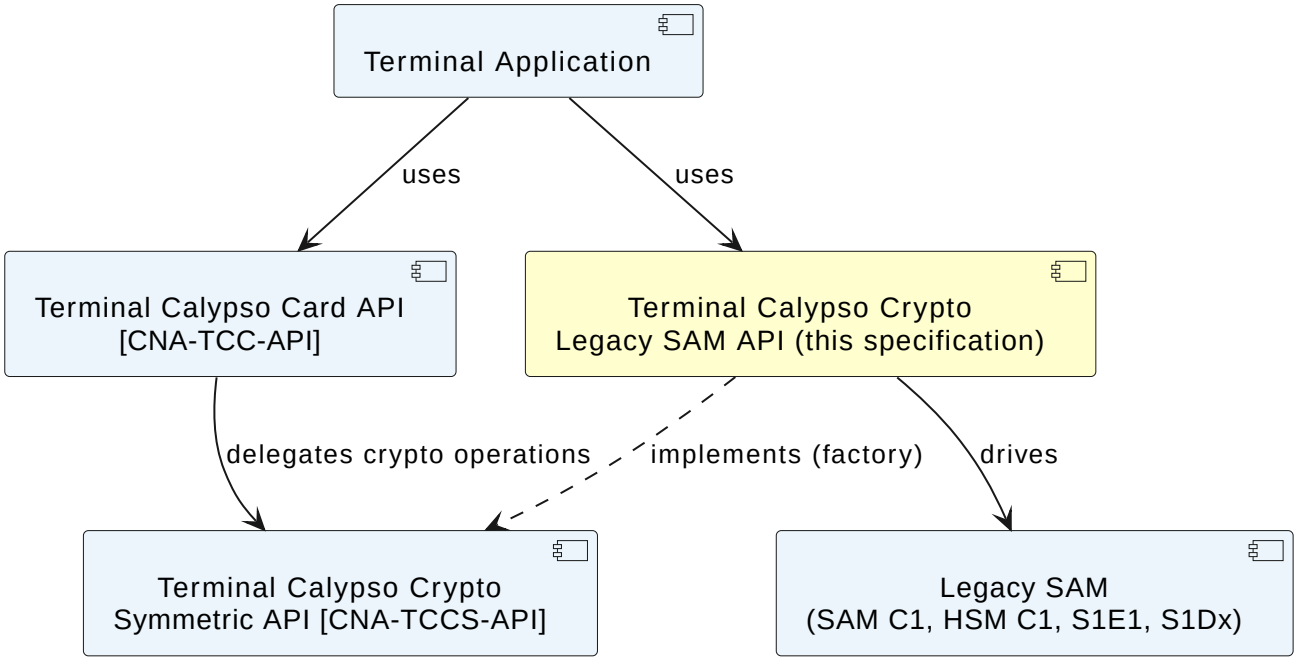

Figure 1. Terminal Calypso Crypto Legacy SAM API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`calypso.crypto.legacysam`	Public API	Factory, properties and shared enumerations (system key types, counter increment access, GetData tags).
`calypso.crypto.legacysam.sam`	Public API	SAM data model: `LegacySam`, `KeyParameter`, `SamParameters`, `LegacySamSelectionExtension`.
`calypso.crypto.legacysam.spi`	SPI	Application-implemented SPIs for unlock data computation and SAM revocation.
`calypso.crypto.legacysam.transaction`	Public API	Transaction managers, signature/key/certificate data carriers, security setting, traceability mode and exceptions.

4.3. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

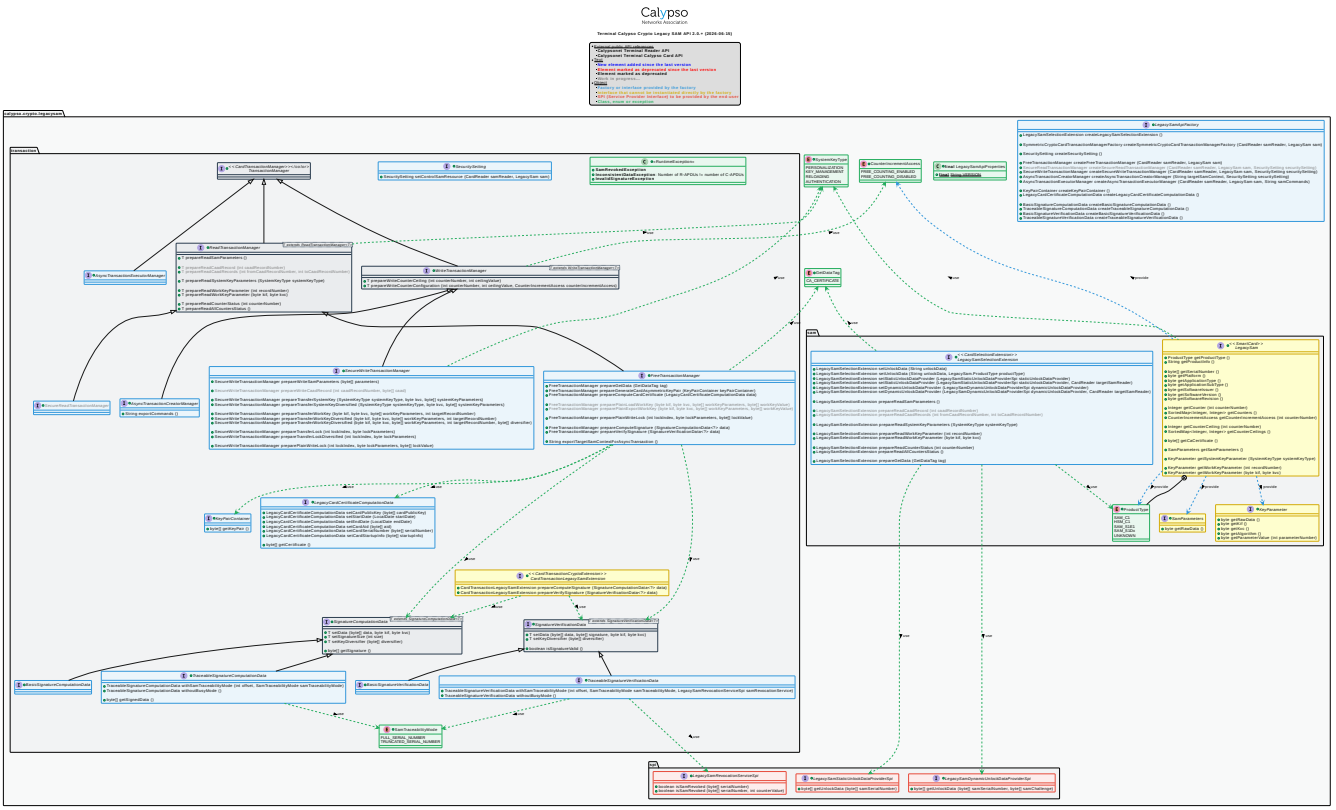

Figure 2. Terminal Calypso Crypto Legacy SAM API — UML Class Diagram

5. API Specification

5.1. Classes

5.1.1. Legacy SAM API Properties

Name	`LegacySamApiProperties`
Kind	Final class
Namespace	`calypso.crypto.legacysam`
Since	0.1
Purpose	Exposes the immutable properties of the Terminal Calypso Crypto Legacy SAM API.

Constants

Name	Type	Since	Description
`VERSION`	`string`	0.1	String representation of the version of the API (e.g. `"2.0"`).

5.2. API Interfaces

5.2.1. Async Transaction Creator Manager

Name	`AsyncTransactionCreatorManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`WriteTransactionManager<AsyncTransactionCreatorManager>`
Since	0.2
Purpose	Prepares write commands using a control SAM and exports them as a serialised stream for later replay by an `AsyncTransactionExecutorManager`. An instance is obtained via `LegacySamApiFactory.createAsyncTransactionCreatorManager`.
See also	`AsyncTransactionExecutorManager`

Export Commands

Signature	`exportCommands() → string`
Since	0.2
Description	Returns a non-empty `string` carrying the prepared commands. These commands can later be imported and processed by an `AsyncTransactionExecutorManager`.
Returns	A non-empty `string`.
Throws	None.

5.2.2. Async Transaction Executor Manager

Name	`AsyncTransactionExecutorManager`
Kind	Interface

Namespace	`calypso.crypto.legacysam.transaction`
Extends	`TransactionManager<AsyncTransactionExecutorManager>`
Since	0.2
Purpose	Executes a command stream prepared by an `AsyncTransactionCreatorManager`. Declares no additional operation. An instance is obtained via `LegacySamApiFactory.createAsyncTransactionExecutorManager`.
See also	`AsyncTransactionCreatorManager`

5.2.3. Basic Signature Computation Data

Name	`BasicSignatureComputationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`SignatureComputationData<BasicSignatureComputationData>`
Since	0.1
Purpose	Specialisation for basic signature computation using the **Data Cipher** command. An instance is obtained via `LegacySamApiFactory.createBasicSignatureComputationData`.

`BasicSignatureComputationData` declares no additional operation.

5.2.4. Basic Signature Verification Data

Name	`BasicSignatureVerificationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`SignatureVerificationData<BasicSignatureVerificationData>`
Since	0.1
Purpose	Specialisation for basic signature verification using the **Data Cipher** command. An instance is obtained via `LegacySamApiFactory.createBasicSignatureVerificationData`.

`BasicSignatureVerificationData` declares no additional operation.

5.2.5. Card Transaction Legacy SAM Extension

Name	`CardTransactionLegacySamExtension`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`CardTransactionCryptoExtension` (CNA-TCC-API)
Since	0.3
Purpose	Card-transaction crypto extension enriching the CNA-TCC-API command set with SAM-backed signature computation and verification. Instances are obtained through `SecureRegularModeTransactionManager.getCryptoExtension` or `SecureExtendedModeTransactionManager.getCryptoExtension`.

Prepare Compute Signature

Signature	`prepareComputeSignature(`
	`  data: SignatureComputationData<?>`
	`) → CardTransactionLegacySamExtension`

Since	0.3
Description	Schedules the execution of a **Data Cipher** or **PSO Compute Signature** command in the context of a Calypso card transaction. Once the command is processed, the result is available in the supplied input/output `BasicSignatureComputationData` or `TraceableSignatureComputationData`. The signature serves many purposes, for example: · **To sign data recorded in a contactless card or ticket.** To speed up processing, it is RECOMMENDED to use a constant signing key (not diversified before ciphering) and to insert the serial number of the card or ticket at the beginning of the data to sign. · **To sign data reported from a terminal to a central system.** The terminal SAM carries a signing work key diversified with its own serial number, which guarantees that the data was indeed signed by this SAM. The central system SAM uses the master signing key, diversified before signing with the diversifier set previously by the **Select Diversifier** command.
Parameters	`data` (`SignatureComputationData`) — the input/output data containing the parameters of the command.
Returns	The current instance (`CardTransactionLegacySamExtension`).
Throws	`IllegalArgumentException` — if the input data is inconsistent.

See also	`SignatureComputationData`
	`BasicSignatureComputationData`
	`TraceableSignatureComputationData`

Prepare Verify Signature

Signature	`prepareVerifySignature(`
	`  data: SignatureVerificationData<?>`
	`) → CardTransactionLegacySamExtension`

Since	0.3
Description	Schedules the execution of a **Data Cipher** or **PSO Verify Signature** command in the context of a Calypso card transaction. Once processed, the result is available in the supplied input/output `BasicSignatureVerificationData` or `TraceableSignatureVerificationData`.
Parameters	`data` (`SignatureVerificationData`) — the input/output data containing the parameters of the command.
Returns	The current instance (`CardTransactionLegacySamExtension`).
Throws	`IllegalArgumentException` — if the input data is inconsistent. `SamRevokedException` — if the signature was computed in **SAM traceability** mode, the revocation check was requested and the SAM is revoked (traceable signatures only).

See also	`SignatureVerificationData`
	`BasicSignatureVerificationData`
	`TraceableSignatureVerificationData`

5.2.6. Free Transaction Manager

Name	`FreeTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`ReadTransactionManager<FreeTransactionManager>`
Since	0.1

Purpose	Transaction manager used when no control SAM is involved. An instance is obtained via `LegacySamApiFactory.createFreeTransactionManager`.

Export Target SAM Context For Async Transaction

Signature	`exportTargetSamContextForAsyncTransaction() → string`
Since	0.2
Description	Executes the commands required to obtain the security context of the target SAM. The returned `string` is passed to `LegacySamApiFactory.createAsyncTransactionCreatorManager`.
Returns	A non-empty `string`.
Throws	None.

Prepare Compute Card Certificate

Signature	`prepareComputeCardCertificate(`
	`  data: LegacyCardCertificateComputationData`
	`) → FreeTransactionManager`

Since	0.5
Description	Schedules the execution of a **PSO Compute Certificate** command. The result is written into the supplied data carrier.
Parameters	`data` (`LegacyCardCertificateComputationData`) — the input/output data containing the parameters of the command.
Returns	The current instance (`FreeTransactionManager`).
Throws	None.
See also	`LegacyCardCertificateComputationData` `LegacySamApiFactory.createLegacyCardCertificateComputationData`

Prepare Compute Signature

Signature	`prepareComputeSignature(`
	`  data: SignatureComputationData<?>`
	`) → FreeTransactionManager`

Since	0.1
Description	Schedules the execution of a **Data Cipher** or **PSO Compute Signature** command. Once the command is processed, the result is available in the supplied input/output `BasicSignatureComputationData` or `TraceableSignatureComputationData`. The signature serves many purposes, for example: · **To sign data recorded in a contactless card or ticket.** To speed up processing, it is RECOMMENDED to use a constant signing key (not diversified before ciphering) and to insert the serial number of the card or ticket at the beginning of the data to sign. · **To sign data reported from a terminal to a central system.** The terminal SAM carries a signing work key diversified with its own serial number, which guarantees that the data was indeed signed by this SAM. The central system SAM uses the master signing key, diversified before signing with the diversifier set previously by the **Select Diversifier** command.
Parameters	`data` (`SignatureComputationData`) — the input/output data containing the parameters of the command.
Returns	The current instance (`FreeTransactionManager`).
Throws	`IllegalArgumentException` — if the input data is inconsistent.

See also	`SignatureComputationData`
	`BasicSignatureComputationData`
	`TraceableSignatureComputationData`
	`LegacySamApiFactory.createBasicSignatureComputationData`

Prepare Generate Card Asymmetric Key Pair

Signature	`prepareGenerateCardAsymmetricKeyPair(`
	`  keyPairContainer: KeyPairContainer`
	`) → FreeTransactionManager`

Since	0.5
Description	Schedules the execution of a **Card Generate Asymmetric Key Pair** command. The result is written into the supplied container.
Parameters	`keyPairContainer` (`KeyPairContainer`) — the container for the output data.
Returns	The current instance (`FreeTransactionManager`).
Throws	None.
See also	`KeyPairContainer` `LegacySamApiFactory.createKeyPairContainer`

Prepare Get Data

Signature	`prepareGetData(tag: GetDataTag) → FreeTransactionManager`
Since	0.5
Description	Schedules the execution of a **Get Data** command for the supplied tag. Once processed, the data is accessible through the dedicated getters of the `LegacySam`, such as `getCaCertificate`.
Parameters	`tag` (`GetDataTag`) — the tag to retrieve the data for.
Returns	The current instance (`FreeTransactionManager`).
Throws	None.

Prepare Plain Write Lock

Signature	`preparePlainWriteLock(`
	`  lockIndex: byte,`
	`  lockParameters: byte,`
	`  lockValue: byte array`
	`) → FreeTransactionManager`

Since	0.7
Description	Schedules the execution of a **Write Key** command to set the lock file of the SAM. The lock value is transferred in plain text.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters. `lockValue` — a 16-byte `byte array` representing the lock's value.
Returns	The current instance (`FreeTransactionManager`).
Throws	`IllegalArgumentException` — if `lockValue` is out of range.

Prepare Verify Signature

Signature	`prepareVerifySignature(`
	`  data: SignatureVerificationData<?>`
	`) → FreeTransactionManager`

Since	0.1
Description	Schedules the execution of a **Data Cipher** or **PSO Verify Signature** command. Once processed, the result is available in the supplied input/output `BasicSignatureVerificationData` or `TraceableSignatureVerificationData`.
Parameters	`data` (`SignatureVerificationData`) — the input/output data containing the parameters of the command.
Returns	The current instance (`FreeTransactionManager`).
Throws	`IllegalArgumentException` — if the input data is inconsistent. `SamRevokedException` — if the signature was computed in **SAM traceability** mode, the revocation check was requested and the SAM is revoked (traceable signatures only).

See also	`SignatureVerificationData`
	`BasicSignatureVerificationData`
	`TraceableSignatureVerificationData`
	`LegacySamApiFactory.createBasicSignatureVerificationData`

5.2.7. Key Pair Container

Name	`KeyPairContainer`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.5
Purpose	Carries the input/output data of `prepareGenerateCardAsymmetricKeyPair`. A key pair consists of a `byte array` holding the public key and the private key values. An instance is obtained via `LegacySamApiFactory.createKeyPairContainer`.

Get Key Pair

Signature	`getKeyPair() → byte array`
Since	0.5
Description	Returns the generated key pair as a 96-byte `byte array`.
Returns	A non-empty `byte array`.
Throws	None.

5.2.8. Key Parameter

Name	`KeyParameter`
Kind	Interface
Namespace	`calypso.crypto.legacysam.sam`
Since	0.2
Purpose	Carries the parameters of a key managed by the SAM (system or work key).

Get Algorithm

Signature	`getAlgorithm() → byte`

Since	0.2
Description	Returns the key algorithm identifier byte.
Returns	A `byte` value.
Throws	None.

Get KIF

Signature	`getKif() → byte`
Since	0.2
Description	Returns the key identifier (KIF).
Returns	A `byte` value.
Throws	None.

Get KVC

Signature	`getKvc() → byte`
Since	0.2
Description	Returns the key version (KVC).
Returns	A `byte` value.
Throws	None.

Get Parameter Value

Signature	`getParameterValue(parameterNumber: integer) → byte`
Since	0.2
Description	Returns the value of the parameter whose number is supplied.
Parameters	`parameterNumber` — the number of the parameter to get (in range `[1..10]`).
Returns	A `byte` value.
Throws	`IllegalArgumentException` — if the argument is out of range.

Get Raw Data

Signature	`getRawData() → byte array`
Since	0.2
Description	Returns the raw key parameter data: 13 bytes carrying KIF, KVC, algorithm and PAR1..PAR10.
Returns	A non-empty `byte array`.
Throws	None.

5.2.9. Legacy Card Certificate Computation Data

Name	`LegacyCardCertificateComputationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.5

Purpose	Carries the input/output data of `prepareComputeCardCertificate`. An instance is obtained via `LegacySamApiFactory.createLegacyCardCertificateComputationData`.

Get Certificate

Signature	`getCertificate() → byte array?`
Since	0.5
Description	Returns the 316-byte certificate generated by the SAM, or `null` if the command has not been processed yet.
Returns	A non-empty `byte array`, or `null` if the command has not been processed yet.
Throws	None.

Set Card AID

Signature	`setCardAid(aid: byte array) → LegacyCardCertificateComputationData`
Since	0.5
Description	Sets the AID of the autonomous PKI application of the target card. The AID MUST be 5 to 16 bytes and MUST NOT contain only zero bytes.
Parameters	`aid` — the AID value as a 5 to 16 bytes byte array. Must not contain only zero bytes.
Returns	The current instance (`LegacyCardCertificateComputationData`).
Throws	`IllegalArgumentException` — if the AID is out of range, or contains only zero bytes.

Set Card Public Key

Signature	`setCardPublicKey(`
	`  cardPublicKey: byte array`
	`) → LegacyCardCertificateComputationData`

Since	0.5
Description	Sets the card public key (64 bytes, secp256r1 curve). This key is used for the verification of card signatures.
Parameters	`cardPublicKey` — the 64-byte array representing the public key on the secp256r1 curve.
Returns	The current instance (`LegacyCardCertificateComputationData`).
Throws	`IllegalArgumentException` — if the key is out of range.

Set Card Serial Number

Signature	`setCardSerialNumber(`
	`  serialNumber: byte array`
	`) → LegacyCardCertificateComputationData`

Since	0.5
Description	Sets the 8-byte card serial number.
Parameters	`serialNumber` — the serial number of the card as an 8-byte byte array.
Returns	The current instance (`LegacyCardCertificateComputationData`).
Throws	`IllegalArgumentException` — if the argument is out of range.

Set Card Startup Info

Signature	`setCardStartupInfo(`
	`  startupInfo: byte array`
	`) → LegacyCardCertificateComputationData`

Since	0.5
Description	Sets the 7-byte startup info to embed in the certificate.
Parameters	`startupInfo` — the 7-byte byte array representing the startup info for the card certificate.
Returns	The current instance (`LegacyCardCertificateComputationData`).
Throws	`IllegalArgumentException` — if the argument is out of range.

Set End Date

Signature	`setEndDate(endDate: date) → LegacyCardCertificateComputationData`
Since	0.5
Description	Sets the end of the validity period. The end date is optional: when it is not defined, the certificate is not subject to an end date constraint.
Parameters	`endDate` — the end date.
Returns	The current instance (`LegacyCardCertificateComputationData`).
Throws	None.

Set Start Date

Signature	`setStartDate(startDate: date) → LegacyCardCertificateComputationData`
Since	0.5
Description	Sets the start of the validity period. The start date is optional: when it is not defined, the certificate is not subject to a start date constraint.
Parameters	`startDate` — the start date.
Returns	The current instance (`LegacyCardCertificateComputationData`).
Throws	None.

5.2.10. Legacy SAM

Name	`LegacySam`
Kind	Interface
Namespace	`calypso.crypto.legacysam.sam`
Extends	`SmartCard` (CNA-TR-API)
Since	0.1
Purpose	Dynamic view of a legacy SAM, updated from selection to the end of the transaction.

A `LegacySam` instance is obtained by casting the CNA-TR-API `SmartCard` produced by the selection scenario when the selection extension was a `LegacySamSelectionExtension`.

As a **dynamic view** of the SAM image, a `LegacySam` is updated after every successful command processed by a **synchronous** transaction manager; **asynchronous** write managers do not update the local `LegacySam`.

Get Application Sub Type

Signature	`getApplicationSubType() → byte`
Since	0.1
Description	Returns the application subtype byte.
Returns	A `byte` value.
Throws	None.

Get Application Type

Signature	`getApplicationType() → byte`
Since	0.1
Description	Returns the application type byte.
Returns	A `byte` value.
Throws	None.

Get CA Certificate

Signature	`getCaCertificate() → byte array?`
Since	0.5
Description	Returns the CA certificate retrieved from the SAM (384 bytes); `null` if not available.
Returns	A non-empty `byte array`, or `null` if not available.
Throws	None.
See also	`LegacySamSelectionExtension.prepareGetData` `FreeTransactionManager.prepareGetData`

Get Counter

Signature	`getCounter(counterNumber: integer) → integer?`
Since	0.1
Description	Returns the value of the supplied counter (`[0..26]`). `null` if the value is not set.
Parameters	`counterNumber` — the number of the counter (in range [0..26]).
Returns	An `integer` value, or `null` if the value is not set.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadCounterStatus`

Get Counter Ceiling

Signature	`getCounterCeiling(counterNumber: integer) → integer?`
Since	0.1
Description	Returns the ceiling value of the supplied counter; `null` if not set.
Parameters	`counterNumber` — the number of the counter ceiling (in range [0..26]).
Returns	An `integer` value, or `null` if not set.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadCounterStatus`

Get Counter Ceilings

Signature	`getCounterCeilings() → sortedMap<integer, integer>`
Since	0.1
Description	Returns the known ceiling values, keyed by ceiling number.
Returns	A `sortedMap<integer, integer>`; empty if no ceiling is known.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadAllCountersStatus`

Get Counter Increment Access

Signature	`getCounterIncrementAccess(counterNumber: integer) → CounterIncrementAccess?`
Since	0.2
Description	Returns the increment-access mode for the supplied counter; `null` if unknown.
Parameters	`counterNumber` — the number of the counter being checked.
Returns	A `CounterIncrementAccess`, or `null` if unknown.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadCounterStatus`

Get Counters

Signature	`getCounters() → sortedMap<integer, integer>`
Since	0.1
Description	Returns the values of the known counters, keyed by counter number.
Returns	A `sortedMap<integer, integer>`; empty if no counter is known.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadAllCountersStatus`

Get Platform

Signature	`getPlatform() → byte`
Since	0.1
Description	Returns the platform identifier byte.
Returns	A `byte` value.
Throws	None.

Get Product Info

Signature	`getProductInfo() → string`
Since	0.1
Description	Returns a textual description of the SAM.
Returns	A non-empty `string`.
Throws	None.

Get Product Type

Signature	`getProductType() → ProductType`
Since	0.1
Description	Returns the SAM product type. Possible values: `SAM_C1`, `HSM_C1`, `SAM_S1E1`, `SAM_S1DX`, `UNKNOWN`.
Returns	A `ProductType`.
Throws	None.

Get SAM Parameters

Signature	`getSamParameters() → SamParameters?`
Since	0.7
Description	Returns the SAM parameters; `null` if not available.
Returns	A `SamParameters`, or `null` if not available.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadSamParameters`

Get Serial Number

Signature	`getSerialNumber() → byte array`
Since	0.1
Description	Returns the SAM serial number.
Returns	A non-empty `byte array`.
Throws	None.

Get Software Issuer

Signature	`getSoftwareIssuer() → byte`
Since	0.1
Description	Returns the software issuer identifier.
Returns	A `byte` value.
Throws	None.

Get Software Revision

Signature	`getSoftwareRevision() → byte`
Since	0.1
Description	Returns the software revision number.
Returns	A `byte` value.
Throws	None.

Get Software Version

Signature	`getSoftwareVersion() → byte`
Since	0.1

Description	Returns the software version number.
Returns	A `byte` value.
Throws	None.

Get System Key Parameter

Signature	`getSystemKeyParameter(systemKeyType: SystemKeyType) → KeyParameter?`
Since	0.2
Description	Returns the parameters of the supplied system key; `null` if not available.
Parameters	`systemKeyType` (`SystemKeyType`) — the system key whose parameters are requested.
Returns	A `KeyParameter`, or `null` if not available.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadSystemKeyParameters`

Get Work Key Parameter (KIF)

Signature	`getWorkKeyParameter(`
	`  kif: byte,`
	`  kvc: byte`
	`) → KeyParameter?`

Since	0.7
Description	Returns the parameters of the work key referenced by its KIF/KVC pair; `null` if not available.
Parameters	`kif` — the key KIF. `kvc` — the key KVC.
Returns	A `KeyParameter`, or `null` if not available.
Throws	None.
See also	`LegacySamSelectionExtension.prepareReadWorkKeyParameters(byte, byte)`

Get Work Key Parameter (Record Number)

Signature	`getWorkKeyParameter(recordNumber: integer) → KeyParameter?`
Since	0.7
Description	Returns the parameters of the work key at the supplied record number (`[1..126]`); `null` if not available.
Parameters	`recordNumber` — the key record number (in range [1..126]).
Returns	A `KeyParameter`, or `null` if not available.
Throws	`IllegalArgumentException` — if the record number is out of range.
See also	`LegacySamSelectionExtension.prepareReadWorkKeyParameters(int)`

5.2.11. Legacy SAM API Factory

Name	`LegacySamApiFactory`
Kind	Interface
Namespace	`calypso.crypto.legacysam`
Since	0.3

Purpose	Factory used by the application to obtain instances of every public type provided by the API.

Create Async Transaction Creator Manager

Signature	`createAsyncTransactionCreatorManager(`
	`  targetSamContext: string,`
	`  securitySetting: SecuritySetting`
	`) → AsyncTransactionCreatorManager`

Since	0.3
Description	Returns a new `AsyncTransactionCreatorManager`. The target SAM context MUST have been produced by `FreeTransactionManager.exportTargetSamContextForAsyncTransaction`.
Parameters	`targetSamContext` — the target SAM context. `securitySetting` (`SecuritySetting`) — the security settings.
Returns	An `AsyncTransactionCreatorManager`.
Throws	`IllegalArgumentException` — if any argument is invalid.

Create Async Transaction Executor Manager

Signature	`createAsyncTransactionExecutorManager(`
	`  samReader: CardReader,`
	`  sam: LegacySam,`
	`  samCommands: string`
	`) → AsyncTransactionExecutorManager`

Since	0.3
Description	Returns a new `AsyncTransactionExecutorManager`. The `samCommands` MUST have been produced by an `AsyncTransactionCreatorManager`.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the SAM image. `samCommands` — a `string` containing the prepared commands.
Returns	An `AsyncTransactionExecutorManager`.
Throws	`IllegalArgumentException` — if any argument is invalid.

Create Basic Signature Computation Data

Signature	`createBasicSignatureComputationData() → BasicSignatureComputationData`
Since	0.3
Description	Creates an empty `BasicSignatureComputationData` that carries the input and output data of a **basic** signature computation performed with the SAM's **Data Cipher** command. For the traceable variant, use `createTraceableSignatureComputationData`.
Returns	A `BasicSignatureComputationData`.
Throws	None.

Create Basic Signature Verification Data

Signature	`createBasicSignatureVerificationData() → BasicSignatureVerificationData`
Since	0.3
Description	Creates an empty `BasicSignatureVerificationData` that carries the input and output data of a **basic** signature verification performed with the SAM's **Data Cipher** command. For the traceable variant, use `createTraceableSignatureVerificationData`.

Returns	A `BasicSignatureVerificationData`.
Throws	None.

Create Free Transaction Manager

Signature	`createFreeTransactionManager(`
	`  samReader: CardReader,`
	`  sam: LegacySam`
	`) → FreeTransactionManager`

Since	0.3
Description	Returns a new `FreeTransactionManager` bound to the supplied SAM and reader.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the SAM image.
Returns	A `FreeTransactionManager`.
Throws	`IllegalArgumentException` — if any argument is invalid.

Create Key Pair Container

Signature	`createKeyPairContainer() → KeyPairContainer`
Since	0.5
Description	Creates an empty `KeyPairContainer` used to collect the output of `prepareGenerateCardAsymmetricKeyPair`.
Returns	A `KeyPairContainer`.
Throws	None.

Create Legacy Card Certificate Computation Data

Signature	`createLegacyCardCertificateComputationData() → LegacyCardCertificateComputationData`
Since	0.5
Description	Creates an empty `LegacyCardCertificateComputationData` that carries the input and output data of `prepareComputeCardCertificate`.
Returns	A `LegacyCardCertificateComputationData`.
Throws	None.

Create Legacy SAM Selection Extension

Signature	`createLegacySamSelectionExtension() → LegacySamSelectionExtension`
Since	0.3
Description	Creates a `LegacySamSelectionExtension` that enriches a CNA-TR-API selection scenario with SAM-specific commands (Unlock, Read Parameters, etc.).
Returns	A `LegacySamSelectionExtension`.
Throws	None.

Create Secure Write Transaction Manager

Signature	`createSecureWriteTransactionManager(`
	`  samReader: CardReader,`
	`  sam: LegacySam,`
	`  securitySetting: SecuritySetting`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Returns a new `SecureWriteTransactionManager` bound to the supplied SAM, reader and security setting.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the SAM image. `securitySetting` (`SecuritySetting`) — the security settings.
Returns	A `SecureWriteTransactionManager`.
Throws	`IllegalArgumentException` — if any argument is invalid.

Create Security Setting

Signature	`createSecuritySetting() → SecuritySetting`
Since	0.3
Description	Creates a `SecuritySetting` carrying the configuration for legacy-SAM transactions secured by a control SAM, as consumed by `createSecureWriteTransactionManager`.
Returns	A `SecuritySetting`.
Throws	None.

Create Symmetric Crypto Card Transaction Manager Factory

Signature	`createSymmetricCryptoCardTransactionManagerFactory(`
	`  samReader: CardReader,`
	`  sam: LegacySam`
	`) → SymmetricCryptoCardTransactionManagerFactory`

Since	0.3
Description	Returns a new factory of CNA-TCCS-API `SymmetricCryptoCardTransactionManager` to be used to secure a Calypso card transaction with the supplied legacy SAM.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the associated control SAM to be used with the card transaction.
Returns	A `SymmetricCryptoCardTransactionManagerFactory` (CNA-TCC-API).
Throws	`IllegalArgumentException` — if any argument is invalid.

Create Traceable Signature Computation Data

Signature	`createTraceableSignatureComputationData() → TraceableSignatureComputationData`
Since	0.3
Description	Creates an empty `TraceableSignatureComputationData` that carries the input and output data of a **traceable** signature computation performed with the SAM's **PSO Compute Signature** command. For the basic variant, use `createBasicSignatureComputationData`.
Returns	A `TraceableSignatureComputationData`.
Throws	None.

Create Traceable Signature Verification Data

Signature	`createTraceableSignatureVerificationData() → TraceableSignatureVerificationData`
Since	0.3
Description	Creates an empty `TraceableSignatureVerificationData` that carries the input and output data of a **traceable** signature verification performed with the SAM's **PSO Verify Signature** command. For the basic variant, use `createBasicSignatureVerificationData`.
Returns	A `TraceableSignatureVerificationData`.
Throws	None.

5.2.12. Legacy SAM Selection Extension

Name	`LegacySamSelectionExtension`
Kind	Interface
Namespace	`calypso.crypto.legacysam.sam`
Extends	`CardSelectionExtension` (CNA-TR-API)
Since	0.3
Purpose	Enriches the CNA-TR-API selection scenario with SAM-specific commands (Unlock, Read Parameters, etc.). An instance is obtained via `LegacySamApiFactory.createLegacySamSelectionExtension`.

If the SAM is **locked**, three mutually exclusive unlocking strategies are offered:

1. The application supplies directly the 16-byte unlock value expected by the SAM (static mode)—see `setUnlockData`.
2. The application supplies a `LegacySamStaticUnlockDataProviderSpi` to compute the unlock value (possibly diversified with the SAM serial number).
3. The application supplies a `LegacySamDynamicUnlockDataProviderSpi` to obtain the 8-byte value expected by the SAM in dynamic mode (computed by an origin SAM).

When the unlocking data is supplied by a provider, a CNA-TR-API `CardReader` is required for additional exchanges. The reader MAY be provided either at selection-extension creation time, or later through the relevant overload (e.g. dynamic SAM reader allocation).

Prepare Get Data

Signature	`prepareGetData(tag: GetDataTag) → LegacySamSelectionExtension`
Since	0.6
Description	Schedules the execution of a **Get Data** command for the supplied tag. The result is accessible via a dedicated getter (e.g. `LegacySam.getCaCertificate`).
Parameters	`tag` (`GetDataTag`)—the tag to retrieve the data for.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	None.

Prepare Read All Counters Status

Signature	`prepareReadAllCountersStatus() → LegacySamSelectionExtension`
Since	0.3
Description	Schedules the reading of the status of every counter.

Returns	The current instance (`LegacySamSelectionExtension`).
Throws	None.

Prepare Read Counter Status

Signature	`prepareReadCounterStatus(counterNumber: integer) → LegacySamSelectionExtension`
Since	0.3
Description	Schedules the execution of **Read Event Counter** and **Read Ceiling** commands to read the status of the supplied counter (`[0..26]`). The actual number of commands sent to the SAM will be reduced by reading the whole record at once.
Parameters	`counterNumber` — the number of the counter whose status is to be read (in range `[0..26]`).
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalArgumentException` — if the counter number is out of range.

Prepare Read SAM Parameters

Signature	`prepareReadSamParameters() → LegacySamSelectionExtension`
Since	0.7
Description	Schedules the execution of a **Read Parameters** command. The result is available via `LegacySam.getSamParameters`.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	None.

Prepare Read System Key Parameters

Signature	`prepareReadSystemKeyParameters(`
	`  systemKeyType: SystemKeyType`
	`) → LegacySamSelectionExtension`

Since	0.3
Description	Schedules the execution of a **Read Key Parameters** command for the supplied system key. The result is available via `LegacySam.getSystemKeyParameter`.
Parameters	`systemKeyType` (`SystemKeyType`) — the type of system key.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	None.

Prepare Read Work Key Parameters (KIF)

Signature	`prepareReadWorkKeyParameters(`
	`  kif: byte,`
	`  kvc: byte`
	`) → LegacySamSelectionExtension`

Since	0.7
Description	Schedules the execution of a **Read Key Parameters** command for the work key referenced by its KIF/KVC pair. Once processed, the result is accessible through `LegacySam.getWorkKeyParameter`.
Parameters	`kif` — the key KIF. `kvc` — the key KVC.
Returns	The current instance (`LegacySamSelectionExtension`).

Throws	None.

Prepare Read Work Key Parameters (Record Number)

Signature	`prepareReadWorkKeyParameters(`
	`  recordNumber: integer`
	`) → LegacySamSelectionExtension`

Since	0.7
Description	Schedules the execution of a **Read Key Parameters** command for the work key at the supplied record number (`[1..126]`). Once processed, the result is accessible through `LegacySam.getWorkKeyParameter`.
Parameters	`recordNumber` — the key record number (in range `[1..126]`).
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalArgumentException` — if the record number is out of range.

Set Dynamic Unlock Data Provider (No Target SAM Reader)

Signature	`setDynamicUnlockDataProvider(`
	`  dynamicUnlockDataProvider: LegacySamDynamicUnlockDataProviderSpi`
	`) → LegacySamSelectionExtension`

Since	0.4
Description	Sets the dynamic-unlock-data provider. This overload is used when the card reader needed to communicate with the target SAM is supplied later in the workflow. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`dynamicUnlockDataProvider` (`LegacySamDynamicUnlockDataProviderSpi`) — an implementation of `LegacySamDynamicUnlockDataProviderSpi`.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalStateException` — if an unlocking setting has already been set.

Set Dynamic Unlock Data Provider (Target SAM Reader)

Signature	`setDynamicUnlockDataProvider(`
	`  dynamicUnlockDataProvider: LegacySamDynamicUnlockDataProviderSpi,`
	`  targetSamReader: CardReader`
	`) → LegacySamSelectionExtension`

Since	0.4
Description	Sets the dynamic-unlock-data provider, with the target SAM reader supplied at creation time. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`dynamicUnlockDataProvider` (`LegacySamDynamicUnlockDataProviderSpi`) — an implementation of `LegacySamDynamicUnlockDataProviderSpi`. `targetSamReader` (`CardReader` (CNA-TR-API)) — the card reader used to communicate with the target SAM.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalStateException` — if an unlocking setting has already been set.

Set Static Unlock Data Provider (No Target SAM Reader)

Signature	`setStaticUnlockDataProvider(`
	`  staticUnlockDataProvider: LegacySamStaticUnlockDataProviderSpi`
	`) → LegacySamSelectionExtension`

Since	0.4
Description	Sets the static-unlock-data provider. To be used when the SAM reader is supplied later in the workflow. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`staticUnlockDataProvider` (`LegacySamStaticUnlockDataProviderSpi`) — an implementation of `LegacySamStaticUnlockDataProviderSpi`.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalStateException` — if an unlocking setting has already been set.

Set Static Unlock Data Provider (Target SAM Reader)

Signature	`setStaticUnlockDataProvider(`
	`  staticUnlockDataProvider: LegacySamStaticUnlockDataProviderSpi,`
	`  targetSamReader: CardReader`
	`) → LegacySamSelectionExtension`

Since	0.4
Description	Sets the static-unlock-data provider, with the target SAM reader supplied at creation time. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`staticUnlockDataProvider` (`LegacySamStaticUnlockDataProviderSpi`) — an implementation of `LegacySamStaticUnlockDataProviderSpi`. `targetSamReader` (`CardReader` (CNA-TR-API)) — the card reader used to communicate with the target SAM.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalStateException` — if an unlocking setting has already been set.

Set Unlock Data (No Product Type)

Signature	`setUnlockData(unlockData: string) → LegacySamSelectionExtension`
Since	0.3
Description	Sets the unlock data (8 or 16 bytes, supplied as a 32-character hexadecimal `string`) used to unlock a SAM C1 and schedules an **Unlock** command in the first position of the selection scenario. The **Unlock** command MUST be executed only after a successful filtering.
Parameters	`unlockData` — the unlock data as a 32-character hexadecimal `string`.
Returns	The current instance (`LegacySamSelectionExtension`).
Throws	`IllegalArgumentException` — if the unlock data is malformed or out of range. `IllegalStateException` — if an unlocking setting has already been set.

Set Unlock Data (Product Type)

Signature	`setUnlockData(`
	`  unlockData: string,`
	`  productType: LegacySam.ProductType`
	`) → LegacySamSelectionExtension`

Since	0.3
Description	Sets the unlock data (8 or 16 bytes, supplied as a 32-character hexadecimal `string`) used to unlock a SAM C1, with an explicit target product type, and schedules an **Unlock** command in the first position of the selection scenario. The **Unlock** command MUST be executed only after a successful filtering.
Parameters	`unlockData` — the unlock data as a 32-character hexadecimal `string`. `productType` (`LegacySam.ProductType`) — the targeted product type.
Returns	The current instance (`LegacySamSelectionExtension`).

Throws	`IllegalArgumentException` — if any argument is malformed or out of range. `IllegalStateException` — if an unlocking setting has already been set.

5.2.13. Read Transaction Manager

Name	`ReadTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Generics	`<T extends ReadTransactionManager<T>>`
Extends	`TransactionManager<T>`
Since	0.1
Purpose	Adds **read**-oriented operations to the common transaction manager: SAM parameters, system/work keys and counter status.

The operations on this interface mirror their equivalents on `LegacySamSelectionExtension`. `prepareReadSamParameters`, `prepareReadSystemKeyParameters`, `prepareReadWorkKeyParameters` (both overloads), `prepareReadCounterStatus` and `prepareReadAllCountersStatus` behaves identically.

5.2.14. SAM Parameters

Name	`SamParameters`
Kind	Interface
Namespace	`calypso.crypto.legacysam.sam`
Since	0.7
Purpose	Carries the parameters of the SAM as a single immutable record.

Get Raw Data

Signature	`getRawData() → byte array`
Since	0.7
Description	Returns the raw data of the SAM parameters as a 29-byte array.
Returns	A non-empty `byte array`.
Throws	None.

5.2.15. Secure Write Transaction Manager

Name	`SecureWriteTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`WriteTransactionManager<SecureWriteTransactionManager>`
Since	0.7
Purpose	**Synchronous** write transaction manager that requires a control SAM. Used to write SAM parameters, transfer keys and transfer lock files. An instance is obtained via `LegacySamApiFactory.createSecureWriteTransactionManager`.

Prepare Plain Write Lock

Signature	`preparePlainWriteLock(`
	`  lockIndex: byte,`
	`  lockParameters: byte,`
	`  lockValue: byte array`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command to set the lock file with a plain-text 16-byte lock value.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters. `lockValue` — a 16-byte byte array representing the lock's value.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	`IllegalArgumentException` — if `lockValue` is out of range.

Prepare Transfer Lock

Signature	`prepareTransferLock(`
	`  lockIndex: byte,`
	`  lockParameters: byte`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command transferring a lock file from the control SAM to the target SAM.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	None.

Prepare Transfer Lock Diversified

Signature	`prepareTransferLockDiversified(`
	`  lockIndex: byte,`
	`  lockParameters: byte`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command transferring a lock file from the control SAM to the target SAM, with diversification by the target SAM serial number.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	None.

Prepare Transfer System Key

Signature	`prepareTransferSystemKey(`
	`  systemKeyType: SystemKeyType,`
	`  kvc: byte,`
	`  systemKeyParameters: byte array`
	`) → SecureWriteTransactionManager`

Since	0.7

Description	Schedules a **Write Key** command transferring a system key from the control SAM to the target SAM. `systemKeyParameters` MUST be 29 bytes long.
Parameters	`systemKeyType` (`SystemKeyType`) — the system key to transfer. `kvc` — the KVC of the key. `systemKeyParameters` — the 29-byte key parameters.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	`IllegalArgumentException` — if any argument is out of range.

Prepare Transfer System Key Diversified

Signature	`prepareTransferSystemKeyDiversified(`
	`  systemKeyType: SystemKeyType,`
	`  kvc: byte,`
	`  systemKeyParameters: byte array`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command transferring a system key from the control SAM to the target SAM, with the key first diversified with the target SAM serial number. `systemKeyParameters` MUST be 29 bytes long.
Parameters	`systemKeyType` (`SystemKeyType`) — the system key to transfer. `kvc` — the KVC of the key. `systemKeyParameters` — the 29-byte key parameters.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	None.

Prepare Transfer Work Key

Signature	`prepareTransferWorkKey(`
	`  kif: byte,`
	`  kvc: byte,`
	`  workKeyParameters: byte array,`
	`  targetRecordNumber: integer`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command transferring a work key from the control SAM to the target SAM. `workKeyParameters` MUST be 29 bytes long. `targetRecordNumber` MUST be in `[0..126]`; `0` means the SAM MUST choose the location.
Parameters	`kif` — the KIF of the key. `kvc` — the KVC of the key. `workKeyParameters` — a 29-byte byte array containing the key parameter data. `targetRecordNumber` — the number of the record where to store the key (in range [0..126]).
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	`IllegalArgumentException` — if any argument is out of range.

Prepare Transfer Work Key Diversified (Diversifier)

Signature	`prepareTransferWorkKeyDiversified(`
	`  kif: byte,`
	`  kvc: byte,`
	`  workKeyParameters: byte array,`
	`  targetRecordNumber: integer,`
	`  diversifier: byte array`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command transferring a work key from the control SAM to the target SAM, with diversification by the supplied 8-byte diversifier. `workKeyParameters` MUST be 29 bytes long. `targetRecordNumber` MUST be in `[0..126]`; `0` means the SAM MUST choose the location.
Parameters	`kif` — the KIF of the key. `kvc` — the KVC of the key. `workKeyParameters` — a 29-byte byte array containing the key parameter data. `targetRecordNumber` — the number of the record where to store the key (in range [0..126]). `diversifier` — a 8-byte byte array to use as key diversifier.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	None.

Prepare Transfer Work Key Diversified (No Diversifier)

Signature	`prepareTransferWorkKeyDiversified(`
	`  kif: byte,`
	`  kvc: byte,`
	`  workKeyParameters: byte array,`
	`  targetRecordNumber: integer`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Key** command transferring a work key from the control SAM to the target SAM, with diversification by the target SAM serial number. `workKeyParameters` MUST be 29 bytes long. `targetRecordNumber` MUST be in `[0..126]`; `0` means the SAM MUST choose the location.
Parameters	`kif` — the KIF of the key. `kvc` — the KVC of the key. `workKeyParameters` — a 29-byte byte array containing the key parameter data. `targetRecordNumber` — the number of the record where to store the key (in range [0..126]).
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	None.

Prepare Write SAM Parameters

Signature	`prepareWriteSamParameters(`
	`  parameters: byte array`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Schedules a **Write Parameters** command. The supplied buffer MUST be 29 bytes long.
Parameters	`parameters` — a 29-byte byte array representing the content of the SAM parameters file.
Returns	The current instance (`SecureWriteTransactionManager`).
Throws	`IllegalArgumentException` — if the argument is out of range.

5.2.16. Security Setting

Name	`SecuritySetting`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.2
Purpose	Security setting carrier for legacy-SAM transactions secured by a control SAM. An instance is obtained via `LegacySamApiFactory.createSecuritySetting`.

Set Control SAM Resource

Signature	`setControlSamResource(`
	`  samReader: CardReader,`
	`  controlSam: LegacySam`
	`) → SecuritySetting`

Since	0.2
Description	Sets the control SAM and the reader through which it can be accessed.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader through which the control SAM is accessed. `controlSam` (`LegacySam`) — the control SAM used to secure the transaction.
Returns	The current instance (`SecuritySetting`).
Throws	`IllegalArgumentException` — if the product type of `controlSam` is `UNKNOWN`.

5.2.17. Signature Computation Data

Name	`SignatureComputationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Generics	`<T extends SignatureComputationData<T>>`
Since	0.1
Purpose	Common base contract for the input/output data of a signature computation.

Get Signature

Signature	`getSignature() → byte array`
Since	0.1
Description	Returns the computed signature (1 to 8 bytes).
Returns	A non-empty `byte array`.
Throws	`IllegalStateException` — if the command has not yet been processed.

Set Data

Signature	`setData(`
	`  data: byte array,`
	`  kif: byte,`
	`  kvc: byte`
	`) → T`

Since	0.1

Description	Sets the data to sign and the KIF/KVC of the key.
Parameters	`data` — the data to be signed. `kif` — the KIF of the key to be used for the signature computation. `kvc` — the KVC of the key to be used for the signature computation.
Returns	The current instance (`T`).
Throws	None.

Set Key Diversifier

Signature	`setKeyDiversifier(diversifier: byte array) → T`
Since	0.1
Description	Sets an explicit 1-to-8-byte key diversifier. By default, the diversifier is the full serial number of the target card or SAM.
Parameters	`diversifier` — the diversifier to be used (from 1 to 8 bytes long).
Returns	The current instance (`T`).
Throws	None.

Set Signature Size

Signature	`setSignatureSize(size: integer) → T`
Since	0.1
Description	Sets the expected signature size in bytes (`[1..8]`). The default size is 8 bytes; the longer the signature, the more secure it is.
Parameters	`size` — the expected size [1..8].
Returns	The current instance (`T`).
Throws	None.

5.2.18. Signature Verification Data

Name	`SignatureVerificationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Generics	`<T extends SignatureVerificationData<T>>`
Since	0.1
Purpose	Common base contract for the input/output data of a signature verification.

Is Signature Valid

Signature	`isSignatureValid() → boolean`
Since	0.1
Description	Indicates whether the signature is valid.
Returns	A `boolean` value.
Throws	`IllegalStateException` — if the command has not yet been processed.

Set Data

Signature	`setData(`
	`  data: byte array,`
	`  signature: byte array,`
	`  kif: byte,`
	`  kvc: byte`
	`) → T`

Since	0.1
Description	Sets the signed data, the signature and the KIF/KVC of the verification key.
Parameters	`data` — the signed data. `signature` — the associated signature. `kif` — the KIF of the key to be used for the signature verification. `kvc` — the KVC of the key to be used for the signature verification.
Returns	The current instance (`T`).
Throws	None.

Set Key Diversifier

Signature	`setKeyDiversifier(diversifier: byte array) → T`
Since	0.1
Description	Sets an explicit 1-to-8-byte key diversifier. Default: full serial number of the target card or SAM.
Parameters	`diversifier` — the diversifier to be used (from 1 to 8 bytes long).
Returns	The current instance (`T`).
Throws	None.

5.2.19. Traceable Signature Computation Data

Name	`TraceableSignatureComputationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`SignatureComputationData<TraceableSignatureComputationData>`
Since	0.1
Purpose	Specialisation for traceable signature computation using the **PSO Compute Signature** command. An instance is obtained via `LegacySamApiFactory.createTraceableSignatureComputationData`.

Get Signed Data

Signature	`getSignedData() → byte array`
Since	0.1
Description	Returns the data that was actually signed. If **SAM traceability** mode was enabled, the returned data embeds the SAM traceability information.
Returns	A non-empty `byte array`.
Throws	`IllegalStateException` — if the command has not yet been processed.

With SAM Traceability Mode

Signature	`withSamTraceabilityMode(`
	`  offset: integer,`
	`  samTraceabilityMode: SamTraceabilityMode`
	`) → TraceableSignatureComputationData`

Since	0.1
Description	Enables the **SAM traceability** mode. The SAM replaces the bits after the supplied offset by its serial number (3 or 4 bytes) followed by the new value (3 bytes) of the counter associated with the signing key. The mode is disabled by default.
Parameters	`offset` — the bit offset after which the traceability data is written. `samTraceabilityMode` (`SamTraceabilityMode`) — the traceability mode to apply.
Returns	The current instance (`TraceableSignatureComputationData`).
Throws	None.

Without Busy Mode

Signature	`withoutBusyMode() → TraceableSignatureComputationData`
Since	0.1
Description	Disables the **Busy** mode. The Busy mode (enabled by default) rejects repeated **PSO Verify Signature** attempts for a few seconds after a failure. Keeping the Busy mode enabled is RECOMMENDED for security-sensitive flows.
Returns	The current instance (`TraceableSignatureComputationData`).
Throws	None.

5.2.20. Traceable Signature Verification Data

Name	`TraceableSignatureVerificationData`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`SignatureVerificationData<TraceableSignatureVerificationData>`
Since	0.1
Purpose	Specialisation for verifications performed with the **PSO Verify Signature** command. An instance is obtained via `LegacySamApiFactory.createTraceableSignatureVerificationData`.

With SAM Traceability Mode

Signature	`withSamTraceabilityMode(`
	`  offset: integer,`
	`  samTraceabilityMode: SamTraceabilityMode,`
	`  samRevocationService: LegacySamRevocationServiceSpi`
	`) → TraceableSignatureVerificationData`

Since	0.1
Description	Indicates that the signature was computed in **SAM traceability** mode. When `samRevocationService` is supplied, the revocation status of the signing SAM is checked through it.
Parameters	`offset` — the bit offset after which the traceability data was written. `samTraceabilityMode` (`SamTraceabilityMode`) — the traceability mode used. `samRevocationService` (`LegacySamRevocationServiceSpi`) — the optional service used to check the signing SAM's revocation status.
Returns	The current instance (`TraceableSignatureVerificationData`).
Throws	None.

See also	`TraceableSignatureComputationData.withSamTraceabilityMode`

Without Busy Mode

Signature	`withoutBusyMode() → TraceableSignatureVerificationData`
Since	0.1
Description	Indicates that the signature was **not** computed in **Busy** mode. By default the signature is assumed to have been computed in Busy mode. When Busy mode is enabled, after a **PSO Verify Signature** fails because of an incorrect signature, the SAM rejects any further **PSO Verify Signature** command issued in Busy mode for a few seconds, answering with the **busy** status word. In that case the application SHOULD repeat the command until the SAM is no longer busy— the busy window lasts a few seconds and never more than ten. Note that after a reset of the SAM, **PSO Verify Signature** commands issued in Busy mode keep failing with the busy status until the end of the busy period.
Returns	The current instance (`TraceableSignatureVerificationData`).
Throws	None.
See also	`TraceableSignatureComputationData.withoutBusyMode`

5.2.21. Transaction Manager

Name	`TransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`CardTransactionManager` (CNA-TR-API)
Since	0.1
Purpose	Common base of every legacy-SAM transaction manager. Provides command processing for the SAM.

The Legacy SAM `TransactionManager` is not generic; processing of prepared commands is inherited from `CardTransactionManager` (CNA-TR-API) via the `processCommands()` operation. The Legacy SAM `TransactionManager` does **not** redefine `processCommands()`; it relies on the inherited contract. Additional failure modes specific to the SAM dialog MAY be reported through `InvalidSignatureException` (signature verification failure) and `InconsistentDataException` (data inconsistency).

5.2.22. Write Transaction Manager

Name	`WriteTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Generics	`<T extends WriteTransactionManager<T>>`
Extends	`TransactionManager<T>`
Since	0.2
Purpose	Common base of every **write** transaction manager. Writes MAY be performed synchronously or asynchronously.

Prepare Write Counter Ceiling

Signature	`prepareWriteCounterCeiling(`
	`  counterNumber: integer,`
	`  ceilingValue: integer`
	`) → T`

Since	0.2
Description	Schedules a **Write Ceilings** command for a single counter. The ceiling value MUST be a positive integer less than or equal to `FFFFFAh`. **Warning:** in an asynchronous transaction, the content of the `LegacySam` object MUST NOT be updated.
Parameters	`counterNumber` — the number of the counter whose ceiling is to be written (in range [0..26]). `ceilingValue` — the desired value for the ceiling. The ceiling value is defined as a positive integer less than or equal to 16777210 (in hexadecimal: FFFFFAh).
Returns	The current instance (`T`).
Throws	`IllegalArgumentException` — if any argument is out of range.

Prepare Write Counter Configuration

Signature	`prepareWriteCounterConfiguration(`
	`  counterNumber: integer,`
	`  ceilingValue: integer,`
	`  counterIncrementAccess: CounterIncrementAccess`
	`) → T`

Since	0.2
Description	Schedules a **Write Ceilings** command for a counter ceiling and its free-incrementation configuration. Because the command writes a full record of nine counters, the application MUST have first read the status of the counters of that record (or have called this method for each of the other eight counters of the same record), otherwise an exception is raised when processing the commands. **Warning:** in the case of an asynchronous transaction, the content of the `LegacySam` is not updated.
Parameters	`counterNumber` — the number of the counter to configure. `ceilingValue` — the ceiling value to set for the counter. `counterIncrementAccess` (`CounterIncrementAccess`) — the free-incrementation access configuration.
Returns	The current instance (`T`).
Throws	`IllegalArgumentException` — if any argument is out of range.

5.3. SPI Interfaces

5.3.1. Legacy SAM Dynamic Unlock Data Provider SPI

Name	`LegacySamDynamicUnlockDataProviderSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.legacysam.spi`
Since	0.4
Purpose	Interface that the application MUST implement to compute the dynamic unlock data expected by the SAM.

Get Unlock Data

Signature	`getUnlockData(`
	`  samSerialNumber: byte array,`
	`  samChallenge: byte array`
	`) → byte array`

Since	0.4
Description	Returns the dynamic unlock data computed by an origin SAM. The serial number and the challenge are required to prepare the **SAM Generate Unlock** command.
Parameters	`samSerialNumber` — the target SAM serial number. `samChallenge` — the challenge provided by the target SAM.
Returns	An 8-byte `byte array`.
Throws	None.

5.3.2. Legacy SAM Revocation Service SPI

Name	`LegacySamRevocationServiceSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.legacysam.spi`
Since	0.1
Purpose	Interface that the application MUST implement to check dynamically whether a SAM is revoked.

Is SAM Revoked (Counter Value)

Signature	`isSamRevoked(`
	`  serialNumber: byte array,`
	`  counterValue: integer`
	`) → boolean`

Since	0.1
Description	Checks if the SAM with the supplied serial number and counter value is revoked. The supplied serial number MAY be complete (4 bytes) or partial (the 3 least significant bytes).
Parameters	`serialNumber` — the complete or partial SAM serial number to check. `counterValue` — the SAM counter value.
Returns	`true` if the SAM is revoked, `false` otherwise.
Throws	None.

Is SAM Revoked (No Counter Value)

Signature	`isSamRevoked(serialNumber: byte array) → boolean`
Since	0.1
Description	Checks if the SAM with the supplied serial number is revoked. The serial number MAY be complete (4 bytes) or partial (3 LSBytes).
Parameters	`serialNumber` — the complete or partial SAM serial number to check.
Returns	`true` if the SAM is revoked, `false` otherwise.
Throws	None.

5.3.3. Legacy SAM Static Unlock Data Provider SPI

Name	`LegacySamStaticUnlockDataProviderSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.legacysam.spi`
Since	0.4
Purpose	Interface that the application MUST implement to compute the static unlock data expected by the target SAM.

Get Unlock Data

Signature	`getUnlockData(samSerialNumber: byte array) → byte array`
Since	0.4
Description	Returns the static unlock data expected by the target SAM. The serial number MAY be used as a diversifier.
Parameters	`samSerialNumber` — the target SAM serial number.
Returns	A 16-byte `byte array`.
Throws	None.

5.4. Enumerations

5.4.1. Counter Increment Access

Name	`CounterIncrementAccess`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam`
Since	0.3
Purpose	Possible access rights for incrementing event counters using the **Increment Counter** command.

Value	Since	Description
`FREE_COUNTING_ENABLED`	0.3	Enables the use of **Increment Counter** on the targeted event counter.
`FREE_COUNTING_DISABLED`	0.3	Forbids the use of **Increment Counter** on the targeted event counter.

5.4.2. Get Data Tag

Name	`GetDataTag`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam`
Since	0.5
Purpose	Output data tags retrievable through the **Get Data** command. MAY NOT be applicable to all products.

Value	Since	Description
`CA_CERTIFICATE`	0.5	CA Certificate (CACert). MAY be unavailable.

5.4.3. Legacy SAM Product Type

Name	`ProductType`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam.sam.LegacySam`
Since	0.1
Purpose	Product type of a legacy SAM.

Value	Since	Description
`SAM_C1`	0.1	SAM C1.
`HSM_C1`	0.1	SAM C1 HSM.
`SAM_S1E1`	0.1	SAM S1E1.
`SAM_S1DX`	0.1	SAM S1Dx.
`UNKNOWN`	0.1	Unidentified SAM.

5.4.4. SAM Traceability Mode

Name	`SamTraceabilityMode`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.3
Purpose	SAM traceability mode used with traceable signature operations.

Value	Since	Description
`FULL_SERIAL_NUMBER`	0.3	Full SAM serial number (4 bytes).
`TRUNCATED_SERIAL_NUMBER`	0.3	Truncated SAM serial number (3 LSBytes).

5.4.5. System Key Type

Name	`SystemKeyType`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam`
Since	0.2
Purpose	System key types of a legacy SAM. Each type corresponds to a specific role.

Value	Since	Description
`PERSONALIZATION`	0.2	Personalisation key — deciphers and authorises the writing of parameters and system keys.
`KEY_MANAGEMENT`	0.2	Work-file key — deciphers and authorises the writing of work keys.
`RELOADING`	0.2	Reloading key — deciphers and authorises the writing of counter ceilings.
`AUTHENTICATION`	0.2	Authentication key — generates the signature of data read from the SAM.

5.5. Exceptions

5.5.1. Inconsistent Data Exception

Name	`InconsistentDataException`
Kind	Runtime exception
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Indicates the detection of inconsistent data.

The inconsistency falls into one of the following cases:

- a **de-synchronisation of the APDU exchanges**, that is, a number of APDU responses different from the number of APDU requests;
- an **inconsistency in the card data**, which can happen for example when the data read outside the secure session differs from the data read inside it.

5.5.2. Invalid Signature Exception

Name	`InvalidSignatureException`
Kind	Runtime exception
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Indicates that a signature is invalid.

5.5.3. SAM Revoked Exception

Name	`SamRevokedException`
Kind	Runtime exception
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Indicates that the SAM is revoked.